Topic 1 · Immutability & Defensive Copy: Worked Guide

[!!!] CORE MUST-KNOW
IMMUTABILITY · 10–14 MARKS

Reference Aliasing, Privacy Leaks & 5-Step Immutability Fix · CROSS-REF SPEC §1.2

WORKED EXAMPLE 1 Breaking & Fixing an "Immutable" Class (Past Exam Archetype)

10 MARKS EXAM Q

Exam Code: `public final class Person { private final String name; private final List<String> hobbies;`
`public Person(String n, List<String> h) { this.name = n; this.hobbies = h; /* Line A */ }`
`public List<String> getHobbies() { return hobbies; /* Line B */ } }`
Tasks: (a) Write Java code showing how a caller mutates **Person**'s internal list. (b) Fix Line A and Line B using Defensive Copying.

STEP 1 (PART A) Exploit Constructor Aliasing (Line A) OR Getter Leak (Line B)

Even though `private final List<String> hobbies` prevents reassigning the pointer, the **ArrayList** object on the Heap is still **mutable**!

- **Attack 1 (Via Constructor Input Reference):**

```
List<String> ext = new ArrayList<>(List.of("Reading")); Person p = new Person("Sarah", ext);
ext.add("Hacked!"); // Mutates p.hobbies directly because this.hobbies == ext (0x4000)!
```

- **Attack 2 (Via Getter Return Reference):**

```
p.getHobbies().clear(); // Directly wipes out internal list via leaked reference!
```

BEFORE FIX Reference Aliasing (Both Pointers Hit Same Heap Address 0×4000)

PRIVACY LEAK!



STEP 2 (PART B) Apply Defensive Copying on Input (Line A) & Output (Line B)

Allocate a **brand-new independent Heap collection** whenever mutable data crosses the class boundary:

- **Line A Fix (Constructor):** `this.hobbies = new ArrayList<>(hobbies);` *(Allocates fresh list at 0×8000)*
- **Line B Fix (Getter):** `return new ArrayList<>(this.hobbies);` *(Returns fresh copy at 0×9000; 0×8000 is 100% shielded!)*

WHY STRING NEEDS NO COPY

String, **Integer**, **Double**, and **LocalDate** are **already immutable** in Java! Sharing their references is 100% safe; only copy mutable objects (**List**, **Date**, arrays, custom objects with setters).

WHY MARK CLASS "FINAL"?

An immutable class MUST be declared **public final class** so no malicious subclass can extend it and override getters to return mutable state!

WORKED EXAMPLE 2 String Constant Pool & Wrapper Cache Trace (== vs .equals)

HIGH-YIELD TRACE

Exam Code Trace: Determine the exact boolean output (**true** or **false**) for each comparison below and justify using JVM Heap vs Constant Pool mechanics.

JAVA CODE SNIPPET	RESULT	STEP-BY-STEP JVM MEMORY JUSTIFICATION

`String a = "OOSD";`
`String b = "OOSD";`
`System.out.print(a == b);`
	true	Both string literals are interned in the **String Constant Pool**. a and b point to the exact same pooled address (`0x1010 == 0x1010`).

`String c = new String("OOSD");` `System.out.print(a == c);`	false	`new String(...)` explicitly allocates a **brand-new object on the main Heap** (`0x2050`). Reference check `0x1010 == 0x2050` is **false**!
`System.out.print(a.equals(c));`	true	`String.equals()` overrides `Object.equals()` to compare character-by-character sequence (`"OOSD"` vs `"OOSD"`).
`Integer x = 100, y = 100;` `System.out.print(x == y);`	true	Autoboxing calls `Integer.valueOf(100)`. JVM caches `Integer` objects in range **[-128, 127]**, returning the same cached address!
`Integer u = 200, v = 200;` `System.out.print(u == v);`	false	`200` is outside **[-128, 127]**! Autoboxing allocates two separate Heap objects (`new Integer(200)`). `u == v` compares addresses → **false**!

WORKED EXAMPLE 3 Pass-By-Value Reference Trap in Methods

CONCEPTUAL CORE

RULE **Java is 100% Pass-By-Value (Copies the Reference Value!)**
`void modify(Student s1, Student s2) { s1.setGpa(4.0f); s2 = new Student("Bob", 2.0f); }`

- **Caller:** `Student a = new Student("Alice", 3.0f); Student b = new Student("Eve", 3.5f); modify(a, b);`
- **Trace on a:** `s1` receives a copy of address `0x5000`. Calling `s1.setGpa(4.0f)` dereferences `0x5000` and mutates Alice's heap object → **`a.getGpa()` becomes 4.0!**
- **Trace on b:** Reassigning `s2 = new Student(...)` only points local variable **s2** to a new address `0x6000`. Caller's variable **b** still points to `0x5004` ("Eve", 3.5) → **b is UNCHANGED!**

ENCAPSULATION DEFINITION (2017 Q1A)

Encapsulation bundles data (attributes) and methods operating on that data into one class, while **Information Hiding** restricts direct external access using **private** fields and controlled **public** getters/setters.

SHALLOW VS DEEP COPY

If a list contains mutable objects (e.g. **List<Student>**), **new ArrayList<>(list)** only copies references (shallow copy)! Full immutability requires cloning each element: **for (Student s : list) copy.add(new Student(s));**

Topic 2 · Polymorphism & Dispatch: Worked Guide

[!!!] CORE MUST-KNOW
DISPATCH TRACE · 8–12 MARKS

Overriding vs Overloading, Static vs Dynamic Binding · CROSS-REF SPEC §2.2

WORKED EXAMPLE 1 Overriding vs Overloading Dispatch Trace (Past Exam Archetype)

8 MARKS EXAM TRAP

Class Hierarchy:

```
class Base { public void print(int x) {
System.out.println("Base int: " + x); }   public void show() {
System.out.println("Base show"); } }
class Derived extends Base { public void print(double x) {
System.out.println("Derived double: " + x); }   @Override
public void show() { System.out.println("Derived show"); } }
```

Execution: `Base obj = new Derived();  obj.show();  obj.print(5);` — Trace exact output & justify!

STEP 1 Identify Static (Declared) Type vs Dynamic (Actual) Type

In `Base obj = new Derived();`:

- **Static Declared Type of `obj` = `Base`** (Known at compile-time by `javac`).
- **Dynamic Actual Type of `obj` = `Derived`** (Actual object living on Heap at runtime).

STEP 2 Trace Call 1: `obj.show();` → Prints `"Derived show"`

1. **Compile-Time Check:** Compiler checks static type `Base`. Does `Base` have `void show()`? YES.
2. **Override Check:** `Derived.show()` has the **exact same signature** (`show()`) as `Base.show()` → **Method Overriding**!
3. **Run-Time Dynamic Binding:** JVM looks at actual heap object (`Derived`) and executes `Derived.show()` → `"Derived show"`.

STEP 3 Trace Call 2: `obj.print(5);` → Prints `"Base int: 5"` (THE TRAP!)

1. **Overload vs Override:** `Derived.print(double)` has a **different parameter type** from `Base.print(int)`. It **OVERLOADS** (does NOT override!) `print`.
2. **Compile-Time Static Binding:** Overload resolution happens 100% at compile-time using ONLY the **Static Type (Base)**!
3. Inside `Base`, the ONLY method is `print(int)`. The compiler hard-binds the call to `Base.print(int)`. Since `print(int)` is NOT overridden in `Derived`, runtime executes `Base.print(5)` → `"Base int: 5"`!

WORKED EXAMPLE 2 Subclass Constructor Chaining & `super()` (2016 Exam Q1)

10 MARKS CODING

> **2016 Exam Q1:** Given immutable `Person(String firstName, String lastName)`, write subclass `Student extends Person` with immutable `String studentID` (initialized once at creation) and mutable `float gpa`.

FULL SCORE CODE Constructor Chaining + Mixed Mutability Pattern

```
public class Student extends Person {
    private final String studentID; // Immutable → final + NO setter
    private float gpa;              // Mutable → private + get/set

    public Student(String fName, String lName, String id, float gpa) {
        super(fName, lName);        // MUST BE LINE 1! Calls Person()
        this.studentID = id;
        this.gpa = gpa;
    }
    public String getStudentID() { return this.studentID; } // Getter
    public float getGpa()        { return this.gpa; }
    public void setGpa(float gpa){ this.gpa = gpa; }        // Setter
}
```

DECISION MATRIX Abstract Class vs Interface Selection Guide

EXAM DESIGN Q

FEATURE / SCENARIO	ABSTRACT CLASS (`EXTENDS`)	INTERFACE (`IMPLEMENTS`)
Semantic Relationship	**"IS-A"** taxonomy (`Doctor`, `Receptionist` extend `Staff`)	**"CAN-DO"** capability across unrelated classes (`Comparable`, `Sortable`)
Instance Variables (State)	**YES:** Can declare `private/protected` non-final instance fields (e.g. `id`, `name`)	**NO:** Cannot hold instance state! All variables are `public static final` constants
Constructors	**YES:** Has constructors invoked via `super(...)` to initialize shared fields	**NO:** Interfaces cannot have constructors and cannot be instantiated
Multiple Inheritance	**Single class inheritance only** (`class A extends B`)	**Multiple type inheritance** (`class A implements I1, I2, I3`)

2-STAGE DISPATCH GOLDEN RULE

Stage 1 (Compile-Time): Compiler uses **Static Type** to pick the method signature (resolves all Overloading).
Stage 2 (Run-Time): JVM uses **Dynamic Type** ONLY if that exact signature was Overridden!

ALWAYS WRITE @OVERRIDE

If the developer wrote `@Override` above `print(double x)` in `Derived`, the compiler would immediately flag an error, catching the accidental overload bug!

IMPLICIT SUPER() COMPILE ERROR

If superclass `Person` defines `Person(String n)`, Java **removes the default no-arg constructor**! If `Student` forgets `super(n)` on Line 1, compilation fails!

SAFE DOWNCASTING PATTERN

Before calling a subclass-only method on a `Person p` reference, ALWAYS check:

```
if (p instanceof Student) {
Student s = (Student) p;
s.setGpa(4.0f); }
```

Topic 3 · UML Class Diagrams: Worked Design Guide

[!!!] CORE MUST-KNOW
UML DIAGRAMMING · 18–24 MARKS

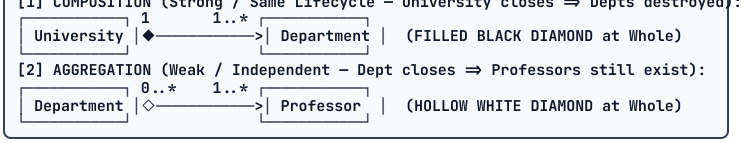
Aggregation vs Composition Diamonds & Text-to-UML Translation ·
CROSS-REF SPEC §3.2

WORKED EXAMPLE 1 Aggregation (Hollow ◇) vs Composition (Solid ◆)

8 MARKS EXAM Q

Past Exam Prompt: Distinguish Aggregation and Composition in (a) visual UML symbol, (b) lifecycle dependency, and (c) classify Pair 1: *University & Department* and Pair 2: *Department & Professor*.

VECTOR NOTATION MAP Whole-Part Diamond Placement & Lifecycle Comparison



STEP-BY-STEP Complete 6-Relationship UML Arrow Dictionary

1. **Inheritance (IS-A):** `Sub` ———▷ `Super` (Solid line, hollow triangle pointing to Superclass).
2. **Realization (Implements):** `Class - - - -▷ «interface»` (Dashed line, hollow triangle).
3. **Composition (Strong HAS-A):** `Whole` ◆———> `Part` (Solid line, **filled black diamond** at Whole).
4. **Aggregation (Weak HAS-A):** `Whole` ◇———> `Part` (Solid line, **hollow white diamond** at Whole).
5. **Directed Association:** `A` ———> `B` (Solid line, open arrow: A holds field of type B).
6. **Dependency (Uses):** `A - - - -→ B` (Dashed line, open arrow: B is a method parameter or local variable).

WORKED EXAMPLE 2 Translating Prose Spec to UML Class Diagram (2016 Q4 / 2017 Q3)

20 MARKS WORKFLOW

2016 Exam Q4 (Medical Practice): *"Practice employs two types of staff: Receptionists and Doctors (all share employeeNumber, name, address, phone; Doctors have regNumber; Receptionists have lastTrainingDate, provider). Patients register (patientNumber, name) and book Appointments with a Doctor (date, attended, treatmentCost). Practice tracks patient count."*

STEP 1 Extract Nouns → Classes & Shared Hierarchy (Inheritance)

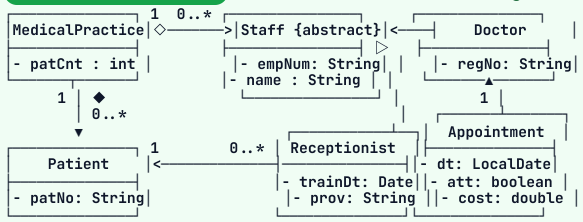
"Two types of staff: Receptionists and Doctors" sharing 4 attributes → Create abstract superclass **`Staff`** with **`- employeeNumber: int`**, **`- name: String`**, etc. Subclasses **`Doctor`** and **`Receptionist`** inherit via ———▷ **`Staff`**!

STEP 2 Spot Association Classes / Many-to-Many Mediators (`Appointment`)

A **`Patient`** books an **`Appointment`** with a **`Doctor`**, and each appointment stores **`date: LocalDate`**, **`attended: boolean`**, **`cost: double`**.

- Model **`Appointment`** as its own class!
- Multiplicities: **`Patient "1" ←—— "0..*" Appointment "0..*" ——> "1" Doctor`** (1 Patient has 0..* Appointments; each Appointment is for exactly 1 Patient and 1 Doctor).

VECTOR ARCHITECTURE Medical Practice UML Class Diagram Blueprint



DIAMOND PLACEMENT TRAP

Students lose marks by drawing the diamond on the *Part* end! **Rule:** The diamond (◇ or ◆) ALWAYS touches the **Whole / Container class** (e.g. **`DeathStar ◆—— DefenceItem`**).

NO REDUNDANT LIST ATTRIBUTES

In UML diagrams, if you draw an association line **`Course ——> 0..* Student`**, do **NOT** also write **`- students: List<Student>`** inside the box (or keep both consistent as taught in tutes).

WHY UML IS USEFUL (2017 Q1B)

1. **Visual Abstraction & Blueprint:** Lets developers spot design flaws (tight coupling, missing classes) BEFORE writing expensive code.
2. **Standardized Communication:** Language-independent notation understood across teams.

COHESION VS COUPLING (2017 Q1D)

- **Cohesion:** How focused a single class's responsibilities are (Want **HIGH**).
- **Coupling:** How heavily classes depend on each other's internals (Want **LOW**).

Topic 4 · SOLID Principles: Worked Refactoring Guide

[!!!] CORE MUST-KNOW
DESIGN CRITIQUE · 10–16 MARKS

Diagnosing Code Smells & Step-by-Step SOLID Refactoring · CROSS-REF SPEC §4.1

WORKED EXAMPLE 1 Diagnosing & Refactoring SOLID Violations (Past Exam Archetype)

10 MARKS CRITIQUE

Exam Code under Critique:

```
public class OrderProcessor { public void processOrder(Order order) {
  if (order.getType().equals("VIP"))
order.setPrice(order.getPrice() * 0.8);
  else if (order.getType().equals("STUDENT"))
order.setPrice(order.getPrice() * 0.9);
  Database.execute("INSERT INTO orders VALUES (" +
order.getId() + ")");
  EmailSender.send(order.getCustomerEmail(), "Order
Confirmed"); } }
```

VIOLATION 1 Single Responsibility Principle (SRP) — "God Method"

- **Why it violates SRP:** `OrderProcessor` has **three unrelated reasons to change**: (1) discount business rules change, (2) database SQL schema changes, or (3) email notification format changes.
- **How to Refactor:** Extract three cohesive classes: `DiscountCalculator` (pricing), `OrderRepository` (database persistence), and `EmailNotificationService` (messaging).

VIOLATION 2 Open/Closed Principle (OCP) — Hardcoded Type Chain

- **Why it violates OCP:** Every time a new order tier is introduced (e.g. `"SENIOR"` or `"STAFF"`), a developer must open and modify `OrderProcessor.processOrder()` to add another `else if` branch, risking breaking existing logic.
- **How to Refactor:** Introduce a `DiscountStrategy` interface with `double applyDiscount(double price)`. Implement `VipDiscount`, `StudentDiscount`. Adding a new tier only requires adding a new class!

VIOLATION 3 Dependency Inversion Principle (DIP) — Static Concrete Coupling

- **Why it violates DIP:** High-level `OrderProcessor` is tightly coupled to low-level concrete `Database` and `EmailSender`.
- **How to Refactor:** Depend on abstractions (`OrderRepository`, `Notifier` interfaces) injected via the constructor!

INSTANT EXAM SMELL DETECTOR

- Long `if/else` on type string or `instanceof` → Violates **OCP** (Fix with Strategy/Polymorphism).
- **Class does Calc + DB + UI/Email** → Violates **SRP** (Split into separate classes).

THREE PILLARS OF GOOD DESIGN (2016 Q5)

1. **Abstraction:** Hide complexity behind interfaces.
2. **Inheritance:** Reuse common base code.
3. **Delegation:** Compose helper objects for flexible runtime behaviour.

WORKED EXAMPLE 2 Clean SOLID Refactored Code & Liskov / ISP Proofs

FULL REFACTORING

AFTER REFACTORING 100% Compliant with SRP + OCP + DIP

```
public interface DiscountStrategy { double computePrice(double raw); }
public class VipDiscount implements DiscountStrategy {
    public double computePrice(double raw) { return raw * 0.8; }
}
public class OrderProcessor {                    // Orchestrator (SRP)
    private final OrderRepository repo;          // Abstraction (DIP)
    private final NotificationService notifier; // Abstraction (DIP)

    public OrderProcessor(OrderRepository r, NotificationService n) {
        this.repo = r; this.notifier = n;
    }
    public void processOrder(Order o, DiscountStrategy d) { // Open (OCP)
        o.setPrice(d.computePrice(o.getPrice()));
        repo.save(o);
        notifier.notifyCustomer(o.getCustomerEmail(), "Confirmed");
    }
}
```

WORKED EXAMPLE 3 Liskov Substitution (LSP) & Interface Segregation (ISP)

CONCEPTUAL MASTERY

PRINCIPLE	CLASSIC VIOLATION CODE EXAMPLE	WHY IT BREAKS & CORRECT OO FIX
Liskov Substitution (LSP)	`class Bird { void fly() {...} } class Penguin extends Bird { void fly() { throw new Error(); } }`	Passing a `Penguin` to a method expecting a `Bird` crashes the program! Subtype violates base contract. **Fix:** Separate `Bird` and interface `Flyable`.
LSP (Rectangle / Square)	`class Square extends Rectangle` Overriding `setWidth(w)` to also set `height = w`.	Client code executing `r.setWidth(5); r.setHeight(4); assert(r.area() == 20);` fails (gets 16)! **Fix:** Both implement immutable **Shape** interface.
Interface Segregation (ISP)	`interface SmartDevice { void print(); void fax(); void scan(); }` Implemented by `BasicPrinter`.	`BasicPrinter` is forced to implement empty/throwing `fax()` and `scan()` methods. **Fix:** Segregate into `Printable`, `Faxable`, `Scannable`.

INHERITANCE VS DELEGATION

Inheritance (White-box reuse): Fixed at compile-time; tightly couples subclass to parent implementation.
Delegation (Black-box reuse): Object forwards call to an interface field; swappable at runtime!

DEPENDENCY INJECTION BENEFIT

Passing `OrderRepository` into the constructor makes **Unit Testing trivial**: in tests, pass a `MockOrderRepository` without touching a real SQL database!

Topic 5 · GoF Design Patterns: Worked Implementation

[!!!] CORE MUST-KNOW PATTERNS · 16-24 MARKS

Strategy, Observer, State, Singleton & Template Method · CROSS-REF SPEC §4.2

WORKED EXAMPLE 1 Strategy Pattern Complete Implementation (2016 Q1g / 2017 Q2)

14-16 MARKS EXAM Q

2016 Exam Q1(3g): *"Course class must display enrolled students using two interchangeable strategies: Name-based Strategy and GPA-based Strategy. Write Java code for the Strategy interface, Concrete Strategies, and Course delegation."*

FULL JAVA CODE 3 Participants: Strategy Interface + Concrete Strategies + Context

```
// 1. Strategy Interface (Defines abstract algorithm contract)
public interface DisplayStrategy {
    void display(List<Student> students);
}
// 2. Concrete Strategies (Encapsulate interchangeable algorithms)
public class NameDisplayStrategy implements DisplayStrategy {
    @Override public void display(List<Student> students) {
        System.out.println("Students ordered by Last Name");
    }
}
public class GpaDisplayStrategy implements DisplayStrategy {
    @Override public void display(List<Student> students) {
        System.out.println("Students ordered by GPA");
    }
}
// 3. Context Class (Holds Strategy reference & delegates execution)
public class Course {
    private ArrayList<Student> students = new ArrayList<>();
    private DisplayStrategy strategy;
    public void setStrategy(DisplayStrategy s) { this.strategy = s; }
    public void displayStudents() {
        if (this.strategy != null) this.strategy.display(students);
    }
}
```

EXAM COMPARISON Strategy Pattern vs Template Method Pattern

- **Strategy (Composition / "HAS-A"):** Swaps the *entire algorithm* at runtime via `setStrategy()`. Uses interface delegation.
- **Template Method (Inheritance / "IS-A"):** Base class defines a `public final void generateReport()` skeleton calling fixed steps (`open()`, `close()`) and `protected abstract void writeBody()` overridden by subclasses at compile-time!

SINGLETON VS PUBLIC STATIC FIELD

Why not `public static Logger instance = new Logger();`?

1. **Lazy Creation:** Singleton creates only when first needed.
2. **Protection:** Public field can be overwritten (`Logger.instance = null`).
3. **Encapsulation:** `getInstance()` can later be changed to return a pool of 2 loggers without breaking callers!

JAVA BUILT-IN PATTERNS

- `java.util.Comparator` → **Strategy Pattern**.
- `java.util.Iterator` → **Iterator Pattern**.
- `AbstractList` / `ArrayList` → **Template Method**.

WORKED EXAMPLE 2 Pattern Selection & Justification Matrix (Past Exam Archetype)

10 MARKS SELECTION

Exam Scenarios: Select the exact GoF pattern and write a 3-point architectural justification:

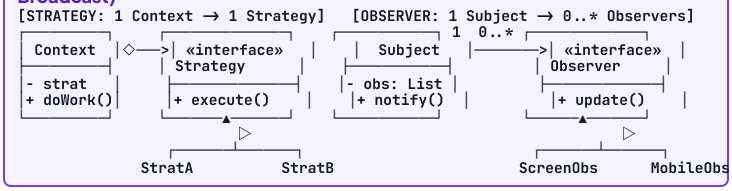
SCENARIO 1 Mokepon Battle Conditions (Asleep, Poisoned, Confused, Normal)

- **Selected Pattern: State Pattern**.
- **Justification:** (1) The Mokepon's turn behaviour changes completely based on its active internal condition; (2) conditions dynamically transition from one to another during battle (e.g., `Asleep` → `Normal`); (3) encapsulating each condition as a `ConditionState` class eliminates massive state-checking `if/else` blocks inside `Mokepon`.

SCENARIO 2 Weather Station / Student Enrolment Broadcasting to Multiple Systems

- **Selected Pattern: Observer Pattern**.
- **Justification:** (1) Establishes a **one-to-many** dependency between Subject (`WeatherStation` / `EnrolmentSystem`) and multiple Observers (`Timetable`, `Billing`, `LMS`); (2) Subject automatically notifies all registered observers on state change without knowing their concrete classes (**Loose Coupling**); (3) new observers can be attached at runtime with zero modifications to Subject (**Open/Closed Principle**).

VECTOR UML MAP Strategy (1-to-1 Algorithm) vs Observer (1-to-Many Broadcast)



STRATEGY VS STATE DISTINCTION

Strategy: Client/caller actively chooses & injects an algorithm (e.g. sort order); strategies don't know about each other.
State: Object's internal state transitions automatically during execution (e.g. Poisoned → Fainted).

WHY USE DESIGN PATTERNS? (2017 Q1C)

1. **Proven Quality & Reusability:** Tested solutions to recurring problems that prevent maintainability bugs.
2. **Shared Vocabulary:** Saying "Observer Pattern" communicates the entire architecture instantly.

Topic 6 · Generics & Collections: Worked Coding Guide

[!!!] CORE MUST-KNOW
JAVA DEVELOPMENT · 20–36 MARKS

Bounded Generics, Nested HashMaps, Comparable & Streams ·

CROSS-REF SPEC §5.1–5.3

WORKED EXAMPLE 1 Bounded Generics & Nested HashMap (2017 Exam Q6 — 20 Marks!)

20 MARKS HARD Q

2017 Exam Q6: Given `interface Categoriser<In, Out> { Out categorise(In input); }`.
(a) Write `StringCategoriser` returning the 1st letter of a String. (b) Write generic `CategorisedMap<K, V, C extends Categoriser<V, K>>` holding a `HashMap<K, ArrayList<V>>` and `put(V value)`, `getCategoryCount(V value)`.

FULL SCORE CODE Complete 20-Mark Solution (2017 Final Exam Q6)

```
// Part (a): Implement Generic Interface with Concrete Types
public class StringCategoriser implements Categoriser<String, String> {
    @Override public String categorise(String input) {
        return input.substring(0, 1);
    }
}
// Part (b) & (c): Bounded Generic Class + Nested HashMap
public class CategorisedMap<K, V, C extends Categoriser<V, K>> {
    private HashMap<K, ArrayList<V>> map;
    private C categoriser;

    public CategorisedMap(C categoriser) {
        this.map = new HashMap<>();
        this.categoriser = categoriser;
    }
    public void put(V value) {
        K key = this.categoriser.categorise(value); // 1. Compute key K
        if (!this.map.containsKey(key)) {           // 2. Init bucket
            this.map.put(key, new ArrayList<V>());
        }
        this.map.get(key).add(value);               // 3. Append value
    }
    public int getCategoryCount(V value) {
        K key = this.categoriser.categorise(value);
        if (!this.map.containsKey(key)) return 0;   // Guard null bucket
        return this.map.get(key).size();
    }
}
```

WORKED EXAMPLE 2 Descending `compareTo` & Best Card Removal (2017 Exam Q4)

- **Descending `Comparable<Card>`:** `public int compareTo(Card other) { return other.getRank().getPoints() - this.getRank().getPoints(); }` *(Note: if question says "c1 worth less than c2 → negative", use `this.points - other.points`!)*
- **Play Best Card (Return & Remove):** `Collections.sort(hand); return hand.remove(hand.size() - 1);` (or index 0 if sorted descending).

GENERIC "EXTENDS" KEYWORD RULE

In a generic type parameter bound `<C extends Categoriser<V, K>>`, Java ALWAYS uses the keyword `extends` even when `Categoriser` is an interface (never write `implements` inside `<...>`)!

SAFE REMOVAL BY ID (2016 Q1E)

To remove a student by `studentID` from an `ArrayList`:

```
Iterator<Student> it = list.iterator();
while(it.hasNext()) {
if(it.next().getId().equals(id))
it.remove(); }
```

WORKED EXAMPLE 3 Java Stream Pipeline Line-by-Line Analysis (2017 Exam Q1f)

4 MARKS STREAM Q

2017 Exam Q1(f): Describe the exact purpose and behaviour of each line in the following stream pipeline:

```
List<Member> result = members.stream()
  .filter(m → m.isActive())   .filter(m → m.getFollowers() > 100000)
  .sorted((m1, m2) → m1.getFollowers() - m2.getFollowers())
.collect(Collectors.toList());
```

PIPELINE STAGE	STREAM OPERATION TYPE & EXACT BEHAVIOUR
`members.stream()`	**Source Stage:** Creates a sequential Stream of `Member` objects from the `members` collection without mutating the original list.
`.filter(m → m.isActive())`	**Intermediate Op 1:** Retains ONLY `Member` elements where `isActive()` returns `true`; discards inactive members.
`.filter(m → m.getFollowers() > 100000)`	**Intermediate Op 2:** Further filters the stream to keep ONLY active members with strictly more than 100,000 followers.
`.sorted((m1, m2) → ...)`	**Stateful Intermediate Op:** Sorts the surviving members in **ascending order** of their follower count using a lambda `Comparator`.
`.collect(Collectors.toList())`	**Terminal Operation:** Triggers pipeline execution and packages the filtered, sorted stream elements into a brand-new `List<Member>` assigned to `result`.

WORKED EXAMPLE 4 Writing Exam Test Cases & Exception Handling (2017 Q3b)

6 MARKS TESTING

TEMPLATE How to Write a Full-Mark Software Test Case in Exam (2017 Q3b)

Every worded test case in SWEN20003 MUST explicitly state **3 components**:
1. **Component / Behaviour Tested:** e.g., *"Testing `DefenceItem.addSoldier()` minimum staffing validation and total staff counter."*
2. **Concrete Test Input:** e.g., *"Create a `LaserTurret` requiring `minSoldiers = 3`, and assign only 2 `Soldier` objects to it."*
3. **Expected Output / Result:** e.g., *"`isOperational()` should return `false` (or throw `InsufficientStaffException`), and count equals 2."*

EXCEPTION RULE Checked vs Unchecked & Catch Block Ordering

- **Checked (`IOException`):** Must be caught or declared in `throws`. • **Unchecked (`NullPointerException`, `IllegalArgumentException`):** Subclasses of `RuntimeException`; indicate programmer bugs.
- **Catch Order:** ALWAYS catch specific subclass (`FileNotFoundException`) BEFORE parent (`IOException`), and `finally` ALWAYS runs!

AUTOMATED VS MANUAL TESTING

Why prefer automated testing (JUnit)?
1. Regression Safety: Runs thousands of tests in seconds after every code refactor.
2. Consistency: Eliminates human fatigue/oversight on boundary edge cases.

HASHMAP EQUALS & HASHCODE

If you use a custom class (e.g. `Pair<K1, K2>` in 2016 Q3) as a `HashMap` key, you MUST override both `equals()` AND `hashCode()` so equal pairs map to the same bucket!